DECA Lab Logbook — Spring Term

Lab 3 | Task 6: Nested Interrupts (int-test3)

Task 6: Nested Interrupts (int-test3)

6.1 Program Overview

The eep1x sheet of lab3i is loaded with int-test3.ram. This program allows INT1 to nest inside the INT0 ISR. Key additions relative to int-test2 are: SETI and CLRI instructions placed inside ISR0, and explicit saving and restoring of PCX.

```
SETI
LOOP  ADD R0, #1
      JNE LOOP
      ORG 0xff00
ISR   MOV R6, Flags    // save flags FIRST (before MOV R5, R0
overwrites flags)
      MOV R5, R0
      LDR R0, [0xFFFE]
      AND R0, #2
      JNE ISR1
ISR0  MOV R0, #1
      STR R0, [0xFFFF]
      MOV R0, #0
      STR R0, [0xFFFF]
      STR R5, [0xFF80]
      STR R6, [0xFF81]
      MOV R0, PCX
      STR R0, [0xFF82]  // save PCX before SETI
      SETI               // re-enable interrupts
      ADD R3, #1
      MOV R0, #3
      SUB R0, #1
      JNE -1
INTEND0
      LDR R0, [0xFF81]
      MOV Flags, R0
      CLRI               // disable interrupts before restore
      LDR R0, [0xFF82]
      MOV PCX, R0        // restore PCX
      LDR R0, [0xFF80]
      RETINT
ISR1  MOV R0, #2
      STR R0, [0xFFFF]
      MOV R0, #0
      STR R0, [0xFFFF]
      ADD R4, #1
INTEND1
      MOV Flags, R6
      MOV R0, R5
      RETINT
```

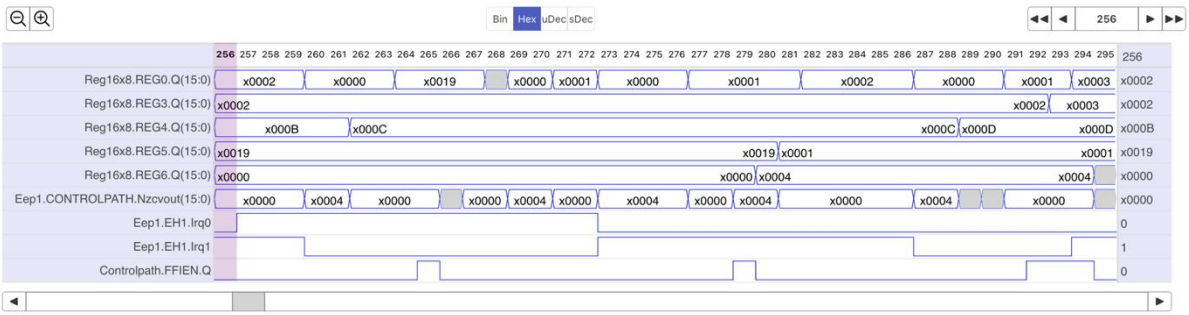
6.2 Simulation

Simulate the eep1x sheet of lab3i running int-test3.ram in the waveform simulator.

The waveform confirms the following sequence of events:

- Cycle 256: IRQ0 = 1. ISR0 starts.
- Cycle 272: IRQ1 = 1. The INT1 interrupt signal is raised.
- Cycle 278: FFIEN = 1 and IRQ1 is accepted, causing a nested entry into ISR1.
- Cycle 288: R4 increments by 1 — ISR1 is executing.
- Cycle 292: R3 increments by 1 — ISR1 has returned and ISR0 continues to run.

This confirms that nested interrupts operate correctly: INT1 is serviced inside INT0, and ISR0 resumes and completes normally afterwards.



6.3 Role of SETI and CLRI Inside the INT0 ISR

Note the placement of SETI and CLRI inside the INT0 ISR. What do these instructions control?

SETI sets the FFIEN (Flip-Flop Interrupt Enable) flag to 1, re-enabling interrupt acceptance. CLRI clears FFIEN to 0, disabling interrupt acceptance.

When the processor enters any ISR, the hardware automatically clears FFIEN to prevent further interrupts until the ISR explicitly re-enables them. In int-test3, SETI is placed inside ISR0 after the critical state has been saved, deliberately re-enabling interrupts to allow INT1 to nest. CLRI is placed later in ISR0, before the state-restoration phase, to prevent any further interrupt from corrupting the restoration process.

6.4 Why SETI Cannot Be Placed Earlier

Explain why correct nesting of INT1 inside INT0 requires both instructions, and why SETI cannot be placed earlier.

EEP1I has only a single hardware PCX register. At the moment ISR0 begins, PCX holds the base program's return address. If SETI were executed before this value had been saved to memory, a nested INT1 would cause the hardware to overwrite PCX with the ISR0 return address, permanently destroying the base program's return address — ISR0 would then have no way to return correctly to the base program.

SETI must therefore only be executed after the sequence MOV R0, PCX → STR R0, [0xFF82] has safely saved the original PCX value to memory.

6.5 Why PCX Must Be Saved in int-test3 but Not in int-test2

Explain why PCX must be saved and later restored in the INT0 ISR here, whereas this was not necessary in int-test2.txt.

int-test2 (non-nested): Interrupts are never nested. The hardware PCX register is written once per

interrupt entry (saving the base program return address) and read once at RETINT. Since ISR0 and ISR1 never overlap, PCX always holds exactly one valid return address and does not need to be saved to memory by software.

int-test3 (nested): INT1 may interrupt INT0 while it is running. When INT1 entry occurs, the hardware writes the current PC (inside ISR0) into PCX, overwriting the base program return address. If ISR0 had not previously saved PCX to memory, the base program return address would be lost. ISR0 must therefore:

- MOV R0, PCX + STR R0, [0xFF82] — save PCX to memory before SETI (before any nested interrupt can occur).
- LDR R0, [0xFF82] + MOV PCX, R0 — restore PCX from memory after CLRI, before RETINT.

This ensures that when ISR0 executes RETINT, PCX contains the correct base program return address, even though it was temporarily overwritten during the nested INT1 service.